
ADJOINT SHARDING FOR VERY LONG CONTEXT TRAINING OF STATE SPACE MODELS

Xingzi Xu ^{1,2*}

Amir Tavanaei ¹

Kavosh Asadi ¹

Karim Bouyarmane ¹

¹ Amazon
Seattle, WA 98109, USA
{xingzixu, atavanaei, kavasadi, bouyarkari}@amazon.com

² Duke University
Durham, NC 27708, USA
xingzi.xu@duke.edu

December 31, 2024

ABSTRACT

Despite very fast progress, efficiently training large language models (LLMs) in very long contexts remains challenging. Existing methods fall back to training LLMs with short contexts (a maximum of a few thousands tokens in training) and use inference time techniques when evaluating on long contexts (above 1M tokens context window at inference). As opposed to long-context-inference, training on very long context input prompts is quickly limited by GPU memory availability and by the prohibitively long training times it requires on state-of-the-art hardware. Meanwhile, many real-life applications require not only inference but also training/fine-tuning with long context on specific tasks. Such applications include, for example, augmenting the context with various sources of raw reference information for fact extraction, fact summarization, or fact reconciliation tasks. We propose adjoint sharding, a novel technique that comprises sharding gradient calculation during training to reduce memory requirements by orders of magnitude, making training on very long context computationally tractable. Adjoint sharding is based on the adjoint method and computes equivalent gradients to backpropagation. We also propose truncated adjoint sharding to speed up the algorithm while maintaining performance. We provide a distributed version, and a paralleled version of adjoint sharding to further speed up training. Empirical results show the proposed adjoint sharding algorithm reduces memory usage by up to 3X with a 1.27B parameter large language model on 1M context length training. This allows to increase the maximum context length during training or fine-tuning of a 1.27B parameter model from 35K tokens to above 100K tokens on a training infrastructure composed of five AWS P4 instances. ³

1 Introduction

Foundation models are a new paradigm in artificial intelligence research focused on building large, general-purpose models that adapt to different tasks [44, 40, 7, 51]. Extensive training on large datasets equips foundation models with broad capabilities, which are then fine-tuned on smaller datasets for specific applications. Foundation models commonly employ the transformer architecture [60]. Despite the immense success, training transformer-based models requires memory growing quadratically with the context length L , limiting their applications on long context tasks [36]. Researchers developed various techniques to conquer this problem, ranging from inference time context window expansion [19, 18], IO-aware algorithms [16, 13, 55], and various linearly scaling language model architectures [23, 15, 49, 6]. On another note, distributed learning enables training large models with a big number of GPUs, and efficient

*Work done during internship at Amazon.

³Additional material for this paper can be found at: <https://adjoint-sharding.github.io>.

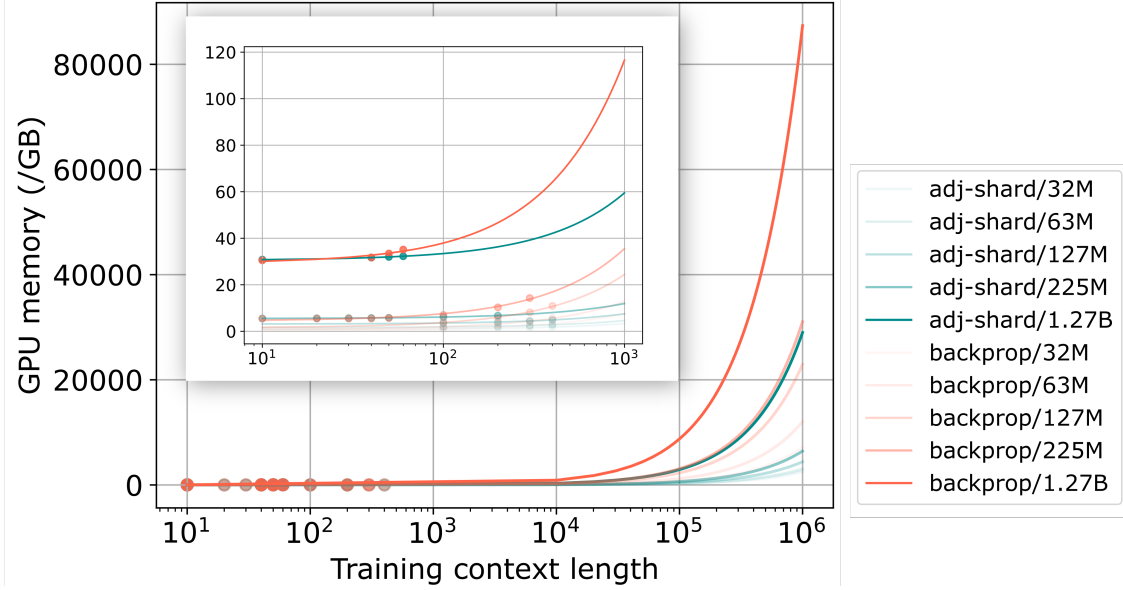


Figure 1: Compared to backpropagation (red lines), adjoint sharding (blue lines) significantly reduces memory requirements at training. Showing memory cost to train 32M, 63M, 127M, 225M, and 1.27B parameter State Space Model (SSM) with batch size 2 and Adam optimizer on one GPU.

training methods like activation checkpointing, model/gradient sharding, and mixed-precision computing have further reduced the memory requirement of training a large model [61, 69, 53, 41, 30]. However, current methodologies are entirely based on backpropagation and compute the gradient as a whole, inevitably requiring a memory growing rapidly with model size and context length [12]. Current sharding methods ignore the activations and only consider the model weights and optimizer states, constituting only a small fraction of the total memory cost [56]. Activation checkpointing is among the limited techniques that consider activation values. Activation checkpointing offloads necessary intermediate states to the CPU and recompute them on the fly, trading compute time for memory reduction [56, 52]. The substantial time required for offloading to the CPU hinders the effectiveness of activation checkpointing.

We propose adjoint sharding to dissemble gradient computation of residual and/or recurrent based models to achieve orders of magnitude lower memory usage during training.

Adjoint method The adjoint sharding method is based on the adjoint method for recurrent models [8, 32]. Given an optimization problem of a parametric recurrent forward process, the adjoint method is concerned with computations of the gradients regarding the process’s parameters. Backpropagation saves intermediate states to calculate gradients, whereas the adjoint method relies on a backward adjoint process to compute gradients. The adjoint method is a constant-memory optimization technique for dynamical systems [9, 66]. In this paper, we are only concerned with the adjoint method for recurrent relations.

Vector-Jacobian product Adjoint sharding dissembles the gradient computation of a large language model (LLM) into independent vector-Jacobian product (VJP) computations. By left-multiplying the Jacobian with a vector, it becomes unnecessary to compute the expensive Jacobian. Modern VJPs are as fast as a forward function call of the model, and can be thousands of times faster than Jacobian computations [2]. We speed up adjoint sharding by employing the VJPs.

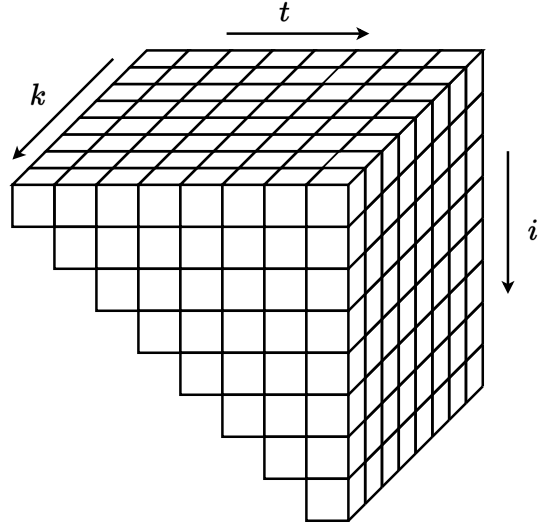


Figure 2: Adjoint sharding dissembles large models’ gradient computations along the sequence dimension t and the layer dimension k . When evaluating the gradient at time t , we perform t vector-Jacobian products along the adjoint dimension i for every layer indices k .

Truncated adjoint sharding Sharding the gradient computation allows us to prioritize the important gradients and disregard the rest, resulting in faster computation. We term this novel method truncated adjoint sharding, and empirically showcase its performance.

Distributed and parallel computation In addition, we have developed a distributed multi-GPU variant of adjoint sharding to further improve the scalability of LLM training. We also analyze the memory cost of parallel computation of adjoint sharding, opening up directions for massive speedups.

State-space models and residual networks Residual networks (ResNets) are a commonly applied neural network structure. We illustrate adjoint sharding assuming a ResNet structure [28]. State-space models (Mamba) have achieved performances on par with attention based models while possessing a linear scaling regarding the context length L , a polynomial speedup compared to the L^2 scaling of transformers [60, 22].

2 Related works

Linear LLMs [17, 5, 49] proposed LLM architectures with a linear inference time complexity. Each of them is formed by stacking K residual layers together, where each layer has a recurrent relation. However, their temporal relationships are nonlinear, which limits the application of adjoint sharding to dissemble the gradients into independent vector-Jacobian products.

Backpropagation through time Applying the adjoint method for recurrent models leads to backpropagation through time (BPTT) [64]. BPTT is a training algorithm developed for recurrent neural networks (RNNs). RNN models suffer from the exploding and vanishing gradient because of the $\prod_{j=i+1}^t \partial \mathbf{f}(\mathbf{x}^j, \mathbf{h}^{j-1}, \mathbf{W}_h) / \partial \mathbf{h}^{i-1}$ term [46]. SSMs provide remedies with careful parameterization of the recurrent dynamics inspired by classical SSM theory [21, 24, 25, 27, 45, 33]. Linear temporal relations allow efficient evaluations of the model, while preserving universal approximation capabilities [63]. By a similar token, truncated adjoint sharding can be seen as a more general version of the truncated backpropagation through time [31, 57].

Neural ordinary differential equations The adjoint method has also been applied to the optimization of continuous systems, especially the ordinary differential equations (ODEs) [9, 20]. Optimizing neural ODEs with autograd requires backpropagating through numerical solvers along every step, using an unrealistic amount of memory. The adjoint method does not backpropagate through the operations of the solver and uses a constant amount of memory. However, applying the adjoint method for continuous systems requires solving a costly ODE initial value problem with dimensionality of the number of parameters.

Low memory training methods Researchers proposed various low memory training techniques to train big models in long contexts. ZERO provides data- and model-parallel training while retaining low communication volume, while eliminating memory redundancies [53]. PyTorch FSDP provides a streamline for model, gradient, and data parallelization [69]. Activation checkpointing discards intermediate values during the forward step, and recompute on the fly during the training phase [56]. CPU offloading scales large model training by offloading data and computations to the CPU, trading computing time for memory reduction [54]. Ring attention leverages the blockwise computation of self-attention and feedforward to distribute long sequences across multiple devices while fully overlapping the communication of key-value blocks with the computation of blockwise attention, enabling very-long context training of attention-based methods [38, 39]. The proposed adjoint sharding distributes state-space model computations across multiple devices as well as multiple multi-GPU-instances (MIG) to enable very-long context training of state-space models.

Context length extension methods Existing context length extension method separate into two classes. The first type is fine-tuning free methods, including Positional Interpolation (PI) [10], the NTKAware Scale ROPE (NTK) [59], and StreamingLLM [65]. The second type is fine-tuning methods, including LongChat [35], LongAlpaca [11], YaRN [50], and LongLlama [11]. Additional methods such as activation beacon do tune a network separate from the LLM [68]. As shown in Figure 3, fine-tuning methods achieve better performances than that of fine-tuning free methods at lengths that they have been fine-tuned on. However, fine-tuning methods suffer from a high computational cost and require a potentially intractable amount of GPU memory during fine-tuning.

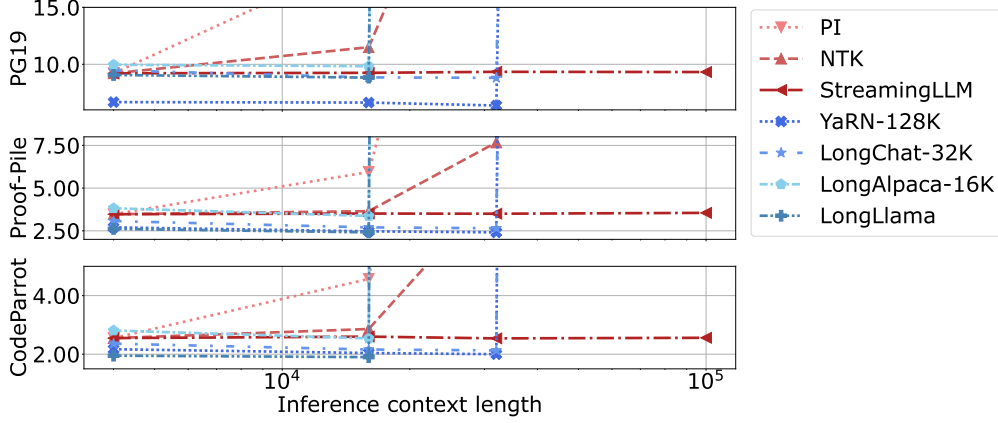


Figure 3: Lines in red are fine-tuning free methods and lines in blue are fine-tuning methods. Fine-tuning methods achieve better performances than fine-tuning free method but often suffer from out of memory issues [10, 59, 65, 35, 11, 50, 68, 58]. Lower values are better across all three tasks.

3 Background

We first give a concise introduction to the state-space models, the residual networks, and the adjoint method.

3.1 State-space models

While our method generally applies to all recurrent models, we illustrate the idea using state-space models (SSMs), which have shown performances at least on par with transformers at small to medium scale [14]. Given an input token sequence $\{\mathbf{x}_t\}_{t=1}^T$, the SSMs first calculate the corresponding matrices $\mathbf{A}^t$, $\mathbf{B}^t$, and $\mathbf{C}^t$ to evolve the dynamics as follows:

$$\mathbf{A}^t = \mathcal{A}(\mathbf{x}^t); \mathbf{B}^t = \mathcal{B}(\mathbf{x}^t); \mathbf{C}^t = \mathcal{C}(\mathbf{x}^t).$$

The SSMs evolve a latent dynamics $\mathbf{h}^t$, whose initial condition $\mathbf{h}^0$ is often assumed to be zero. With $\mathbf{h}^0$ and $\mathbf{A}^t$, $\mathbf{B}^t$ defined, the dynamics evolves as:

$$\mathbf{h}^t = \mathbf{A}^t \mathbf{h}^{t-1} + \mathbf{B}^t \mathbf{x}^t.$$

The matrices $\mathbf{C}^t$ then maps the latent dynamics $\mathbf{h}^t$ back to token space as $\mathbf{y}^t = \mathbf{C}^t \mathbf{h}^t$, with $\mathbf{y}^t$ being the predicted token at t . For a sequence of T tokens, we denote:

$$\begin{aligned} \mathbf{A} &= (\mathbf{A}^1, \mathbf{A}^2, \dots, \mathbf{A}^T), \mathbf{B} = (\mathbf{B}^1, \mathbf{B}^2, \dots, \mathbf{B}^T), \mathbf{C} = (\mathbf{C}^1, \mathbf{C}^2, \dots, \mathbf{C}^T), \\ \mathbf{H} &= (\mathbf{h}^1, \mathbf{h}^2, \dots, \mathbf{h}^T), \mathbf{X} = (\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^T), \mathbf{Y} = (\mathbf{y}^1, \mathbf{y}^2, \dots, \mathbf{y}^T). \end{aligned}$$

In the most general case, we have $\mathbf{H} \in \mathbb{R}^{T \times N}$, $\mathbf{A} \in \mathbb{R}^{T \times N \times N}$, $\mathbf{B} \in \mathbb{R}^{T \times N \times P}$, $\mathbf{C} \in \mathbb{R}^{T \times P \times N}$, $\mathbf{X} \in \mathbb{R}^{T \times P}$, $\mathbf{Y} \in \mathbb{R}^{T \times P}$, where N is the hidden state dimension, and P is the input/output dimension. We evolve the dynamics for $t = 1, \dots, T$, and assume that $\mathbf{h}^0$ is a fixed and predefined constant.

The input to an SSM is $\mathbf{X}$ and $\mathbf{h}^0$, and the output is $\mathbf{Y}$. We define $\text{SSM}(\cdot)$ as performing the following five steps:

1. $\{\mathbf{A}^t\}_{t=1}^T = \{\mathcal{A}(\mathbf{x}^t)\}_{t=1}^T$,
2. $\{\mathbf{B}^t\}_{t=1}^T = \{\mathcal{B}(\mathbf{x}^t)\}_{t=1}^T$,
3. $\{\mathbf{C}^t\}_{t=1}^T = \{\mathcal{C}(\mathbf{x}^t)\}_{t=1}^T$,
4. $\{\mathbf{h}^t\}_{t=1}^T = \{\mathbf{A}^t \mathbf{h}^{t-1} + \mathbf{B}^t \mathbf{x}^t\}_{t=1}^T$;
5. $\{\mathbf{y}^t\}_{t=1}^T = \{\mathbf{C}^t \mathbf{h}^t\}_{t=1}^T$.

The input to the five steps is $\mathbf{X}$, and the output is $\mathbf{Y}$. We can then write $\text{SSM}(\mathbf{X}) = \mathbf{Y}$. SSMs decrease the quadratic computational complexity with sequence length on transformers to linear and decrease the large inference-time memory requirements from the key-value cache. SSM-based models at a small to medium scale have shown performances on par with or better than transformer-based models. For instance, [51, 1] shows that SSM-based mixture-of-experts

(MOE) model outperforms baseline transformer-based MOE model on model sizes as big as 2400M parameters. [62] performed an extensive empirical study and found that while SSMs outperform transformers on various tasks, they underperform on tasks which require strong copying, in-context learning, or long-context reasoning abilities. [62] also experimented with a SSM-transformer hybrid model, which outperforms transformers and is up to eight times faster when generating tokens at inference time. [37] trained a 52B parameter model and further affirmed the hybrid models performances.

3.2 Residual Networks

In practice, we have K SSMs stacked together, and we have a large language head (LLH) $\Omega \in \mathbb{R}^{\mathbb{T} \times P}$, where $\mathbb{T}$ is the number of all possible tokens. To predict a token, we have $\mathbf{o}^t = \Omega \hat{\mathbf{y}}_K^t$. Define $(\mathbf{y}_K^1, \dots, \mathbf{y}_K^T) = \mathbf{Y}_K$, a ResNet computes $\mathbf{Y}_K$ as follows:

$$\begin{aligned} (\mathbf{y}_K^1, \dots, \mathbf{y}_K^T) &= \mathbf{Y}_{K-1} + \text{SSM}_K(\hat{\mathbf{Y}}_{K-1}) \\ &= \mathbf{Y}_0 + \text{SSM}_1(\hat{\mathbf{Y}}_0) + \dots + \text{SSM}_K(\hat{\mathbf{Y}}_{K-1}) \\ &= \mathbf{Y}_0 + \sum_{k=1}^K \text{SSM}_k(\hat{\mathbf{Y}}_{k-1}) = \mathbf{Y}_0 + \sum_{k=1}^K \tilde{\mathbf{Y}}_k, \end{aligned}$$

where $\hat{\mathbf{Y}}_k = (\hat{\mathbf{y}}_k^1, \dots, \hat{\mathbf{y}}_k^T) = (\text{Norm}(\mathbf{y}_k^1), \dots, \text{Norm}(\mathbf{y}_k^T))$ and $\text{SSM}_k(\hat{\mathbf{Y}}_{k-1}) = \tilde{\mathbf{Y}}_k$. Therefore, for a latent state at time t we have $\mathbf{y}_K^t = \mathbf{y}_0^t + \sum_{k=1}^K \tilde{\mathbf{y}}_k^t$.

ResNet has been the foundation of numerous modern networks, including the transformers, diffusion models, segmentation models, SSMs, and more [29, 26, 34, 48]. ResNet’s residual structure allows for a separation between gradients of each layer by applying differentiation on summations.

3.3 Adjoint method

The adjoint method is concerned with optimizing $\mathbf{y}(\mathbf{h}(\boldsymbol{\theta}), \boldsymbol{\theta})$ with respect to $\boldsymbol{\theta}$, where $\mathbf{h}(\boldsymbol{\theta}) \in \mathbb{R}^P$ is the solution to $\mathbf{f}(\mathbf{h}(\boldsymbol{\theta}), \boldsymbol{\theta}) = 0$ [8]. To employ gradient based algorithms like the stochastic gradient descent (SGD) or the Adam, we compute the derivative of $\mathbf{y}$ regarding $\boldsymbol{\theta} \in \mathbb{R}^{|\boldsymbol{\theta}|}$:

$$\frac{d\mathbf{y}}{d\boldsymbol{\theta}} = \frac{\partial \mathbf{y}}{\partial \boldsymbol{\theta}} + \frac{\partial \mathbf{y}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \boldsymbol{\theta}}, \quad (1)$$

with d being the total derivative, and ∂ being the partial derivative. The adjoint method converts computing $d\mathbf{y}/d\boldsymbol{\theta}$ to solving an adjoint equation. In our case, we need the adjoint method for recurrence relations, where $\mathbf{y}$ is given by $\mathbf{y} = \mathbf{y}^t \equiv \mathbf{y}(\mathbf{h}^t(\boldsymbol{\theta}), \boldsymbol{\theta})$, and $\mathbf{h}$ is given by

$$\begin{cases} \mathbf{h}^0 &= \mathbf{b}(\boldsymbol{\theta}), \\ \mathbf{h}^t &= \mathbf{f}(t, \mathbf{h}^{t-1}, \boldsymbol{\theta}). \end{cases} \quad (2)$$

We have

$$\frac{d\mathbf{f}(t, \mathbf{h}^{t-1}, \boldsymbol{\theta})}{d\boldsymbol{\theta}} = \frac{\partial \mathbf{f}(t, \mathbf{h}^{t-1}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} + \frac{\partial \mathbf{f}(t, \mathbf{h}^{t-1}, \boldsymbol{\theta})}{\partial \mathbf{h}^{t-1}} \frac{\partial \mathbf{h}^{t-1}}{\partial \boldsymbol{\theta}}. \quad (3)$$

Proposition 1 [8] *When the states $\mathbf{h}$ are defined as Equation 2, the gradient of $\mathbf{y}$ with respect to $\boldsymbol{\theta}$ is given as:*

$$\begin{cases} d\mathbf{y}^t/d\boldsymbol{\theta} &= \partial \mathbf{y}^t/\partial \boldsymbol{\theta} + \boldsymbol{\lambda}^0 \mathbf{b}(\boldsymbol{\theta}) + \sum_{i=1}^t \boldsymbol{\lambda}^i (\partial \mathbf{f}(i, \mathbf{h}^{i-1}, \boldsymbol{\theta})/\partial \boldsymbol{\theta}), \\ \boldsymbol{\lambda}^t &= \partial \mathbf{y}^t/\partial \mathbf{h}^t, \\ \boldsymbol{\lambda}^{i-1} &= \boldsymbol{\lambda}^i (\partial \mathbf{f}(i, \mathbf{h}^{i-1}, \boldsymbol{\theta})/\partial \mathbf{h}^{i-1}). \end{cases} \quad (4)$$

Equivalently, we have $\boldsymbol{\lambda}^i = (\partial \mathbf{y}^t/\partial \mathbf{h}^t) \left(\prod_{j=t}^{i+1} (\partial \mathbf{f}(j, \mathbf{h}^{j-1}, \boldsymbol{\theta})/\partial \mathbf{h}^{j-1}) \right)$ [32].

After computing adjoint states $\{\boldsymbol{\lambda}^i\}_{i=0}^t$, the computation of the elements of $\boldsymbol{\lambda}^i (\partial \mathbf{f}(i, \mathbf{h}^{i-1}, \boldsymbol{\theta})/\partial \boldsymbol{\theta})$ are independent, allowing parallelism. This computation is a vector-Jacobian product (vjp), with $\boldsymbol{\lambda}^i$ as the vector and $\partial \mathbf{f}(i, \mathbf{h}^{i-1}, \boldsymbol{\theta})/\partial \boldsymbol{\theta}$ as the Jacobian. vjps can be evaluated with the reverse-mode automatic differentiation and initializing the reverse phase with $\boldsymbol{\lambda}^i$ [3]. As each vjp only requires saving their corresponding computation graph, and can be disposed

after the computation, we can compute vjps in parallel on modern GPUs. We will discuss this in more details in subsection 4.5. Adjoint sharding aims to use the adjoint method to replace backpropagation, which solves:

$$\frac{dy^t}{d\theta} = \frac{\partial y^t}{\partial \theta} + \frac{\partial y^t}{\partial \mathbf{h}^t} \left(\frac{\partial f(t, \mathbf{h}^{t-1}, \theta)}{\partial \theta} + \frac{\partial f(t, \mathbf{h}^{t-1}, \theta)}{\partial \mathbf{h}^{t-1}} \left[\frac{\partial f(t-1, \mathbf{h}^{t-2}, \theta)}{\partial \theta} + \frac{\partial f(t-1, \mathbf{h}^{t-2}, \theta)}{\partial \mathbf{h}^{t-2}} \left\{ \frac{\partial f(t-2, \mathbf{h}^{t-3}, \theta)}{\partial \theta} + \dots \right\} \right] \right).$$

The backpropagation requires a sequential accumulation of the gradients, computing from the outmost layer inwards, therefore needs to save the computation graph for computations at all time t 's and creates memory bottlenecks.

4 Adjoint sharding

We now introduce the adjoint sharding technique. We first illustrate the method assuming only one layer of SSM, and generalize to K layers.

4.1 Adjoint sharding for one SSM

Large scale neural networks are usually trained with the autograd framework [4, 47]. However, this framework suffers from a high memory cost when used with networks of recurrent nature [4]. Although activation checkpointing has been developed, which discards part of the intermediate values and recomputes them later on the fly, the memory cost is still high [30]. We employ the adjoint method for recurrence relations to further reduce the memory cost, and more importantly, to break the temporal dependencies of activations and parallelize their computations.

Define $\theta = \langle \theta_{\mathcal{A}}, \theta_{\mathcal{B}}, \theta_{\mathcal{C}} \rangle$ as $\mathcal{A}$'s, $\mathcal{B}$'s, and $\mathcal{C}$'s parameters, for loss $l^t = l(\mathbf{y}^t)$, in the context of a single-layer SSM, we prove:

Proposition 2 *The gradient $dl^t/d\theta$ is given as*

$$\frac{dl^t}{d\theta} = \left[\sum_{i=1}^t \text{vjp}_{\mathcal{A}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1} \right) \right] \oplus \left[\sum_{i=1}^t \text{vjp}_{\mathcal{B}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i \right) \right] \oplus \text{vjp}_{\mathcal{C}^t} \left(\frac{dl^t}{dy^t} \otimes \mathbf{h}^t \right), \quad (5)$$

where the adjoint state $\lambda^{t,\tau} = \mathbf{C}^t (\prod_{i=1}^{t-\tau} \mathbf{A}^{t+1-i})$, $\text{vjp}_{\text{Net}^i}(v) = v \cdot \text{Net}_{\theta}(\text{Input}^i)$, with θ being Net's parameters and i being the index of Input, $\otimes$ is the vector outer product, and $\oplus$ is vector concatenation.

The proof of proposition 2 is in section A.1. The gradient for parameters of $\mathcal{A}$, and $\mathcal{B}$ are each separated into $\{\text{vjp}_{\mathcal{A}^i}(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1})\}_{i=1}^t$, $\{\text{vjp}_{\mathcal{B}^i}(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i)\}_{i=1}^t$, and the gradient for parameters of $\mathcal{C}$ only depend on inputs at time t . After computing the adjoint states, these vjp computations are separate from each other on both the network and the temporal level.

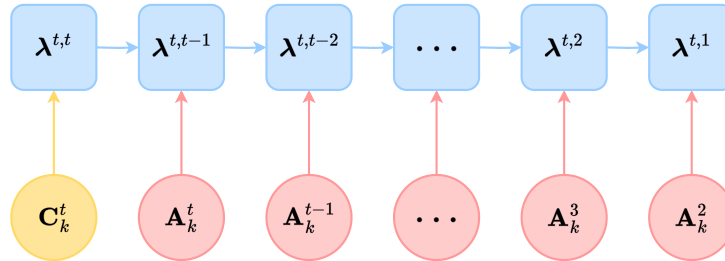


Figure 4: The adjoint states are computed sequentially backwards.

4.2 Adjoint sharding for multiple SSMs

We now generalize the results from subsection 4.1 to the general case of K SSMs concatenated together. As introduced in subsection 3.2, the outputs of each SSM layer are added to the results of the last layer and normalized before it is fed into the next layer. Define the loss over all token predictions $L = \sum_{t=1}^T l^t$, using the residual structure we have

$$\frac{dL}{d\theta} = \sum_{t=1}^T \frac{dl^t}{dy_K^t} \frac{dy_K^t}{d\theta} = \sum_{t=1}^T \frac{dl^t}{dy_K^t} \frac{d(\mathbf{y}_0^t + \sum_{k=1}^K \tilde{\mathbf{y}}_k^t)}{d\theta} = \sum_{t=1}^T \frac{dl^t}{dy_K^t} \sum_{k=1}^K \frac{d\tilde{\mathbf{y}}_k^t}{d\theta}.$$

Combining with proposition 2, we have

Proposition 3 The gradient of the total loss L with respect to the SSM parameters θ is given as

$$\begin{aligned} \frac{dL}{d\theta} = & \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) \right) \\ & \oplus \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right) \\ & \oplus \left(\sum_{t=1}^T \sum_{k=1}^K \text{vjp}_{\mathcal{C}_k^t} \left(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t \right) \right), \end{aligned} \quad (6)$$

where the input to $\text{vjp}_{\mathcal{C}_k^t}(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t)$, $\text{vjp}_{\mathcal{A}_k^i}(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1})$, and $\text{vjp}_{\mathcal{B}_k^i}(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i)$ are computed with the k -th SSM and the $\hat{\mathbf{y}}_{k-1}^i = \text{Norm}(\mathbf{y}_{k-2}^i + \text{SSM}_{k-1}(\hat{\mathbf{Y}}_{k-2}^i))$ (the normalized output sequence of the $(k-1)$ -th SSM). The adjoint state at layer k is defined as $\lambda_k^{t,\tau} = \mathbf{C}_k^t (\prod_{i=1}^{t-\tau} \mathbf{A}_k^{t+1-i})$.

We provide the proof to proposition 3 in section A.2. Define $\Lambda_k^t = \{\lambda_k^{t,\tau}\}_{\tau=1}^t$, proposition 3 shows that the gradients of each network's parameters computed with each token only correlate through the adjoint states $\{\Lambda_k^t\}_{k,t=1,1}^{K,T}$. The adjoint states can be easily computed after a forward pass. The adjoint states can also be computed on the fly in the gradient computation phase, as it only depends on $\mathbf{C}_k^t$ and $\mathbf{A}_k^t$ and has no dependencies on the network Jacobians regarding the network parameters. The adjoint sharding method breaks down the backpropagation computation both layer-wise and token-wise into foundational vjp computations that do not have any dependencies on each other.

We show a schematic of the computations to $dl^t/d\theta_{\mathcal{A}_k}$, $dl^t/d\theta_{\mathcal{B}_k}$, and $dl^t/d\theta_{\mathcal{C}_k}$ in Figure 5 and a schematic for computing the adjoint states in Figure 4.

4.3 Truncated adjoint sharding

One limitation of adjoint sharding is that the number of vjps performed increases polynomially regarding the number of tokens T . In particular, adjoint sharding computes the vjp for $\mathcal{A}_k$ and $\mathcal{B}_k$ $(1+T)T/2$ times, and for $\mathcal{C}_k$ T times. When training large networks with many layers and long context length T , applying adjoint sharding becomes computationally expensive. We propose truncated adjoint sharding, with which we argue that we can get similar results by computing a linearly growing number of vjps, and empirically showcase its performance.

Attention mechanisms have suffered from the $\mathcal{O}(T^2)$ complexities arising from the self-attention structure [60]. To enable training with longer context lengths, global-local attention has been proposed, where we divide the contexts into sections, and compute the attention between sections rather than tokens [67]. [57] proposed truncated backpropagation through time (T-BPTT) to avoid gradient explosion/vanishing when training with long contexts by only counting a fixed number of state transitions. Here, inspired by global-local attention and T-BPTT, instead of computing the full gradient given in Equation 11, we propose to train the SSMs to depend on up to $\bar{T}$ states:

$$\begin{aligned} \frac{dL}{d\theta} = & \left(\sum_{t=1}^T \sum_{k=1}^K \text{vjp}_{\mathcal{C}_k^t} \left(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t \right) \right) \\ & \oplus \left(\sum_{t=1}^{\bar{T}} \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) + \sum_{t=\bar{T}+1}^T \sum_{k=1}^K \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) \right) \\ & \oplus \left(\sum_{t=1}^{\bar{T}} \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) + \sum_{t=\bar{T}+1}^T \sum_{k=1}^K \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right) \end{aligned} \quad (7)$$

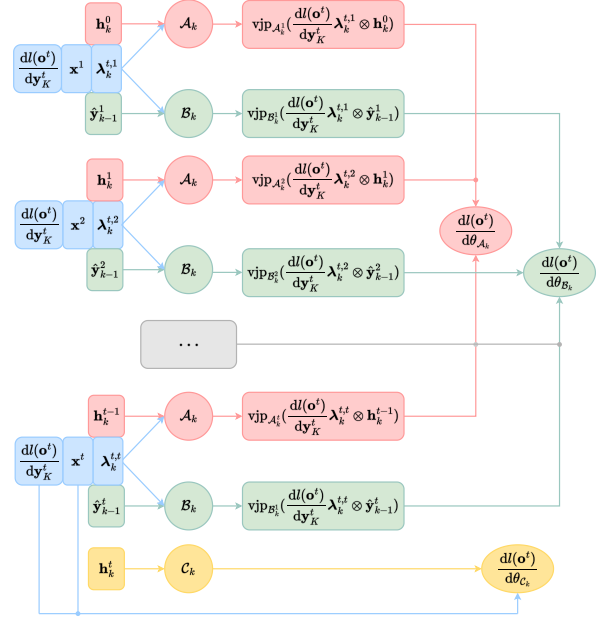


Figure 5: Computation schematic of $dl^t/d\theta_{\mathcal{A}_k}$, $dl^t/d\theta_{\mathcal{B}_k}$, and $dl^t/d\theta_{\mathcal{C}_k}$.

As shown in Equation 7 above, we perform the same computations for $t = 1, \dots, \bar{T}$ as before, and only perform the vjps back to the last $\bar{T}$ states for $t > \bar{T}$. With truncated adjoint sharding, we perform $\bar{T}T + \bar{T}(\bar{T} - 1)/2$ vjps, which grows linearly. We show the number of vjps performed with and without truncated adjoint sharding in Figure 6. When $T = 2000$, truncated adjoint sharding reduces 64% of the vjps when training with a context length of 10K.

The essence of the truncated adjoint sharding method is that we only explicitly count gradients related to the last $\bar{T}$ states. As each state depends on its prior state, states still implicitly depend on all their prior states. We leave investigation of $\bar{T}$'s impact on performances for future works.

4.4 Distributed training

We now discuss how to distribute the storage and compute of the adjoint sharding method, assuming that we have Υ GPUs. Given the networks $\{\mathcal{A}_k, \mathcal{B}_k, \mathcal{C}_k\}_{k=1}^K$, initial tokens $\{\hat{\mathbf{y}}_0^t\}_{t=1}^T = \{\text{Norm}(\mathbf{x}^t)\}_{t=1}^T$, and initial conditions $\{\mathbf{h}_k^0\}_{k=1}^K$ (usually set to $\mathbf{0}$), we can call algorithm 1 to get all necessary vectors for computing the gradient with adjoint sharding.

Algorithm 1 Forward step in evaluation mode on a distributed system

- 1: **Inputs:** $\{\hat{\mathbf{y}}_0^t\}_{t=1}^T, \{\mathbf{h}_k^0\}_{k=1}^K, \{\mathcal{A}_k, \mathcal{B}_k, \mathcal{C}_k\}_{k=1}^K, \Omega$
 - 2: On devices $v = 1, \dots, \Upsilon$, in parallel **do**
 - 3: **for** SSM model index $k = (v - 1)(K/\Upsilon) + 1, \dots, v(K/\Upsilon)$ **do**
 - 4: **for** Time step index $t = 1, \dots, T$ **do**
 - 5: Compute: $\mathbf{A}_k^t = \mathcal{A}_k(\hat{\mathbf{y}}_{k-1}^t); \mathbf{B}_k^t = \mathcal{B}_k(\hat{\mathbf{y}}_{k-1}^t); \mathbf{C}_k^t = \mathcal{C}_k(\hat{\mathbf{y}}_{k-1}^t); \mathbf{h}_k^t = \mathbf{A}_k^t \mathbf{h}_k^{t-1} + \mathbf{B}_k^t \hat{\mathbf{y}}_{k-1}^t; \mathbf{y}_k^t = \mathbf{C}_k^t \mathbf{h}_k^t$.
 - 6: Compute: $\mathbf{y}_k^t = \mathbf{y}_{k-1}^t + \hat{\mathbf{y}}_k^t$.
 - 7: Compute: $\hat{\mathbf{y}}_k^t = \text{Norm}(\mathbf{y}_k^t)$.
 - 8: **end for**
 - 9: **end for**
 - 10: Store: $\{\mathbf{h}_k^t\}_{(t,k)=(1,(v-1)(K/\Upsilon)+1)}^{T,v(K/\Upsilon)}, \{\mathbf{C}_k^t\}_{(t,k)=(1,(v-1)(K/\Upsilon)+1)}^{T,v(K/\Upsilon)}, \{\hat{\mathbf{y}}_k^t\}_{(t,k)=(1,(v-1)(K/\Upsilon)+1)}^{T,v(K/\Upsilon)-1}$,
 $\{\mathbf{A}_k^t\}_{(t,k)=(2,(v-1)(K/\Upsilon)+1)}^{T,v(K/\Upsilon)}$ on device v .
 - 11: Pass: $\{\mathbf{y}_{v(K/\Upsilon)-1}^t\}_{t=1}^T, \{\hat{\mathbf{y}}_{v(K/\Upsilon)-1}^t\}_{t=1}^T$ to device $v + 1$
 - 12: **for** Time step index $t = 1, \dots, T$ **do**
 - 13: Compute: $\{\mathbf{o}^t = \Omega \mathbf{y}_K^t\}_{t=1}^T, \{l(\mathbf{o}^t)\}, \{\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t}\}_{t=1}^T$.
 - 14: **end for**
 - 15: Store: $\{\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t}\}_{t=1}^T$ on all Υ devices.
-

Algorithm 2 Evaluating adjoint states for token index t and ResNet index k with truncated adjoint sharding $\bar{T}$

- 1: **Inputs:** $t, k, \bar{T}, \mathbf{C}_k^t, \{\mathbf{A}_k^i\}_{i=t+2-\bar{T}}^t$
 - 2: Initialize adjoint state $\lambda_k^{t,t} = \mathbf{C}_k^t$
 - 3: Compute: intermediate values:
 - 4: $\zeta^{\bar{T}} = (\mathbf{A}_k^t \mathbf{A}_k^{t-1} \dots \mathbf{A}_k^{t+2-\bar{T}}, \mathbf{A}_k^t \mathbf{A}_k^{t-1} \dots \mathbf{A}_k^{t+3-\bar{T}}, \dots, \mathbf{A}_k^t \mathbf{A}_k^{t-1}, \mathbf{A}_k^t, \mathbb{I})$.
 - 5: Compute: adjoint states $\bar{\Lambda}_k^{\bar{T}} = (\lambda_k^{t,t+1-\bar{T}}, \lambda_k^{t,t+2-\bar{T}}, \dots, \lambda_k^{t,t}) = \mathbf{C}_k^t \zeta^{\bar{T}}$.
 - 6: **Return:** $\bar{\Lambda}_k^{\bar{T}}$.
-

As shown in algorithm 3, to compute the vjps' for token index t and ResNet index k , we only need $t, k, dl(\mathbf{o}^t)/d\mathbf{y}_K^t, \{\mathbf{h}_k^i\}_{i=0}^t, \mathbf{C}_k^t, \{\hat{\mathbf{y}}_{k-1}^i\}_{i=1}^t, \{\mathbf{A}_k^i\}_{i=2}^t$. To compute all the gradients for layer k , we only need $\mathbf{A}$, $\mathbf{h}$, and $\mathbf{C}$ from the k -th layer, and $\hat{\mathbf{y}}$ from the $k - 1$ -th layer. Therefore, we can divide the K layers into Υ pieces, as shown in the appendix A.4.

As the computations are fully independent and we compute the gradients using only data on local devices, we additionally distribute the model and the gradients, as shown in Table 6, where θ_k represents the parameters of $\mathcal{A}_k, \mathcal{B}_k$, and $\mathcal{C}_k$, and Gradient_k represents the optimizer states for θ_k .

The complete training streamline is then as shown in algorithm 4. We fully distribute the activations, computations, gradients, and optimization states across Υ devices. While the forward evaluation pass results across different devices,

Algorithm 3 Evaluating the vjp's for token index t and ResNet index k with truncated adjoint sharding $\bar{T}$

- 1: **Inputs:** $t, k, \bar{T}, \frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t}, \{\mathbf{h}_k^i\}_{i=t-\bar{T}}^t, \mathbf{C}_k^t, \{\mathbf{y}_{k-1}^i\}_{i=t+1-\bar{T}}^t, \{\mathbf{A}_k^i\}_{i=t+2-\bar{T}}^t$
 - 2: Call alg. 2 to compute $\{\lambda_k^{t,i}\}_{i=t+1-\bar{T}}^t$
 - 3: Compute: $\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \otimes \mathbf{h}_k^t, \{\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1}\}_{i=t+1-\bar{T}}^t, \{\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i\}_{i=t+1-\bar{T}}^t$
 - 4: Compute: $\left(\text{vjp}_{\mathbf{C}_k^t} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \otimes \mathbf{h}_k^t \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{A}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{B}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right)$
 - 5: **Return:** $\left(\text{vjp}_{\mathbf{C}_k^t} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \otimes \mathbf{h}_k^t \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{A}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{B}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right)$
-

Algorithm 4 Evaluating $\frac{dL}{d\theta}$ with truncated adjoint sharding $\bar{T}$ on Υ devices

- 1: **Inputs:** $\{\mathbf{y}_0^t\}_{t=1}^T, \{\mathbf{h}_k^0\}_{k=1}^K, \{\mathcal{A}_k, \mathcal{B}_k, \mathcal{C}_k\}_{k=1}^K, \Omega, \bar{T}, \Upsilon$
 - 2: Call alg. 1 for $\{\mathbf{A}_k^t, \mathbf{C}_k^t, \mathbf{h}_k^t, \hat{\mathbf{y}}_k^t\}_{(t,k)=(1,1)}^{(T,K)}, \{\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t}\}_{t=1}^T$ and saved on each GPU device.
 - 3: On each device v , in parallel **do**
 - 4: Initialize gradient $\frac{dL}{d\theta}$
 - 5: **for** Time step index $t = 1, \dots, \bar{T}$, layer index $k = (v-1)(K/\Upsilon) + 1, \dots, v(K/\Upsilon)$ **do**
 - 6: Call alg. 3 for $\Xi = \left(\text{vjp}_{\mathbf{C}_k^t} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \otimes \mathbf{h}_k^t \right), \sum_{i=1}^t \text{vjp}_{\mathbf{A}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right), \sum_{i=1}^t \text{vjp}_{\mathbf{B}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right)$
 - 7: Compute: $\frac{dL}{d\theta} + = \Xi$
 - 8: **end for**
 - 9: **for** Time step index $t = \bar{T} + 1, \dots, T$, layer index $k = (v-1)(K/\Upsilon) + 1, \dots, v(K/\Upsilon)$ **do**
 - 10: Call alg. 3 for $\Xi = \left(\text{vjp}_{\mathbf{C}_k^t} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \otimes \mathbf{h}_k^t \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{A}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right), \sum_{i=t+1-\bar{T}}^t \text{vjp}_{\mathbf{B}_k^i} \left(\frac{dl(\mathbf{o}^t)}{d\mathbf{y}_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right)$
 - 11: Compute: $\frac{dL}{d\theta} + = \Xi$
 - 12: **end for**
 - 13: **Return:** $\frac{dL}{d\theta}$
-

as shown in algorithm 1, the computation of gradients is parallel across the Υ devices. This will speed up the training as the gradient computation takes most of the computation budget. We will also get a memory per GPU close to Mem/Υ , with Mem being the memory cost if we only have a single GPU. If we have $\Upsilon > K$ devices, we can further speed up the forward evaluation by first evaluating $\mathcal{A}, \mathcal{B}, \mathcal{C}$ in parallel, and then sequentially add them together on the distributed devices.

4.5 Parallel computing

Adjoint sharding converts the sequential process of backpropagation gradient computation into individual independent vjps, allowing for parallel computation. We analyze the time and memory cost of $\text{vjp}_{\mathcal{A}_k^i}((dl^t/d\mathbf{y}_K^t)\lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1})$, $\text{vjp}_{\mathcal{B}_k^i}((dl^t/d\mathbf{y}_K^t)\lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i)$, and $\text{vjp}_{\mathcal{C}_k^t}((dl^t/d\mathbf{y}_K^t) \otimes \mathbf{h}_k^t)$.

vjp has a similar time complexity as a forward pass, and a memory complexity of $\text{bs}(|\theta| + \odot) + |\theta|$, where bs is the batch size, $\odot$ is the number of elements in the network output, and $|\theta|$ is the number of parameters [42]. We provide the memory and FLOPs required to compute the vjps in Table 1 [43].

We analyze training with a dataset containing contexts of lengths T , with Υ NVIDIA H100 GPUs, and performing computations in FP16. We use a selective diagonal SSM with K layers, and each $\mathcal{A}_k, \mathcal{B}_k$, and $\mathcal{C}_k$ network is a single-layer multi-layer perceptron (MLP).

For each data point $\{\mathbf{x}^t\}_{t=1}^T$, we store $\{\mathbf{A}_k^t, \mathbf{C}_k^t, \mathbf{h}_k^t, \mathbf{y}_k^t\}_{(t,k)=(1,1)}^{(T,K)}$ and $\{dl(\mathbf{o}^t)/d\mathbf{y}_K^t\}_{t=1}^T$, which is $TK(2N+P)+TP$ FP16 numbers. We also save $\theta_{\mathcal{A}}, \theta_{\mathcal{B}}$, and $\theta_{\mathcal{C}}$, each taking $PN + N$ FP16 numbers. We need to store $T(2NK + PK + P) + 3N(P + 1)$ FP16 numbers before computing the vjp.

		$\text{vjp}_{\mathcal{A}}$	$\text{vjp}_{\mathcal{B}}$	$\text{vjp}_{\mathcal{C}}$
Unstructured SSM	Memory	$\text{bs}(N^2 + \theta_{\mathcal{A}} ^*) + \theta_{\mathcal{A}} $	$\text{bs}(NP + \theta_{\mathcal{B}} ^*) + \theta_{\mathcal{B}} $	$\text{bs}(NP + \theta_{\mathcal{C}} ^*) + \theta_{\mathcal{C}} $
	FLOPs	$\text{bs}(N^2(2P + 1))$	$\text{bs}(NP(2P + 1))$	$\text{bs}(NP \times (2P + 1))$
Diagonal SSM	Memory	$\text{bs}(N + \theta_{\mathcal{A}} ^*) + \theta_{\mathcal{A}} $	$\text{bs}(N + \theta_{\mathcal{B}} ^*) + \theta_{\mathcal{B}} $	$\text{bs}(N + \theta_{\mathcal{C}} ^*) + \theta_{\mathcal{C}} $
	FLOPs	$\text{bs}(N(2P + 1))$	$\text{bs}(N(2P + 1))$	$\text{bs}(N(2P + 1))$
Scalar SSM	Memory	$\text{bs}(1 + \theta_{\mathcal{A}} ^*) + \theta_{\mathcal{A}} $	$\text{bs}(N + \theta_{\mathcal{B}} ^*) + \theta_{\mathcal{B}} $	$\text{bs}(N + \theta_{\mathcal{C}} ^*) + \theta_{\mathcal{C}} $
	FLOPs	$\text{bs}(2P + 1)$	$\text{bs}(N(2P + 1))$	$\text{bs}(N(2P + 1))$

Table 1: Memory and FLOPs required to compute the vjps. $|\theta_{\mathcal{A}}|^*$, $|\theta_{\mathcal{B}}|^*$, and $|\theta_{\mathcal{C}}|^*$ represents the number of elements of the biggest parameter vector of $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$.

As computing all adjoint state sequences takes up to $N(2P + 1)(1 + T)T/2$ FLOPs, it takes $NP(1 + T)/T$ FLOPs on average for each adjoint state. For T large enough, $(1 + T)/T \approx 1$, and we approximate the average FLOPs for each adjoint state with NP . Each vjp then takes $\text{bs}(7NP + 3N)$ FLOPs of computation.

When computing with a selective diagonal SSM with $P = 128$, $N = 225$, and $\text{bs} = 8$, while storing and performing computations in FP16, computing $\text{vjp}_{\mathcal{A}}$, $\text{vjp}_{\mathcal{B}}$, and $\text{vjp}_{\mathcal{C}}$ each takes around 0.6MB memory and 1798144 FLOPs. The capacity of a modern GPU is mostly characterized by FLOPs/sec, which measures the computation speed; GPU memory bandwidth, which is the rate at which a GPU can move data between its memory and processing cores; GPU Memory, which is the amount of data a GPU can hold; and number of Multi-Instance GPU (MIG) instances, which is the number of fully isolated GPU instances with its own high-bandwidth memory, cache, and compute cores a GPU can host.

An NVIDIA H100 Tensor Core GPU has a GPU memory bandwidth 3.35TB/s and performs 1,979 tera FP16 FLOPs per second. Therefore, the memory bandwidth allows computing $(3.35\text{TB/s})/0.6\text{MB} = 5.58 \times 10^6$ batches of vjps per second, and the computing speed allows computing $(1979\text{tera/s})/1798144 = 3.76 \times 1.1\text{E}9$ batches of vjps per second. At the same time, since the H100 GPU has 80GB memory, it can hold up to $80\text{GB}/(0.6\text{MB}/\text{vjp}) = 133$ batches of vjps at the same time if we do not consider any memory overhead. As each H100 GPU can hold up to 7 instances in parallel, we perform the adjoint sharding algorithm with 77 instances, offering as much as a 56x speedup on one AWS P4 instance (8 H100 GPUs). Such speedup cannot be achieved for backpropagation because of its sequential nature.

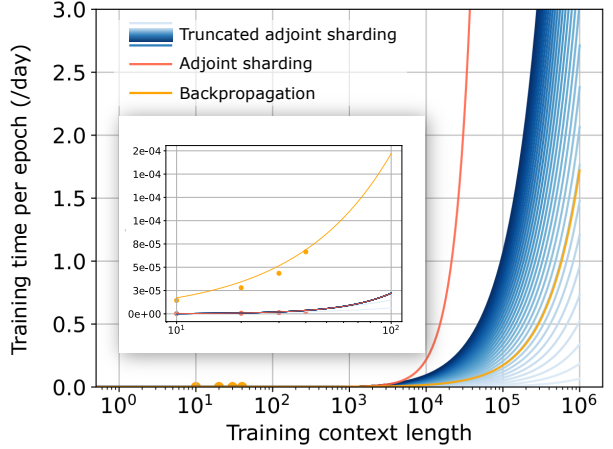


Figure 6: Training time (/day) per epoch comparison for adjoint sharding, truncated adjoint sharding, and backpropagation with different context lengths. Assumed a 100-layer SSM-ResNet model, a 280x acceleration for adjoint sharding from parallel computing (achievable with five Amazon P4 instances), and T from 15 to 2500.

Limitation The adjoint sharding method provides an alternative method of computing gradients to backpropagation. While we analytically proved that the gradients computed from adjoint sharding equals to that from backpropagation, adjoint sharding suffer from a time complexity polynomial regarding the training context length when computing equivalent gradients. We provided the truncated adjoint sharding as a linear time complexity alternative, and leave the analysis of its convergence and further improvements on it for future works. We also provided a distributed and parallel computing algorithm for performing adjoint sharding. However, the overhead of naïve implementation of such algorithm with multi-threading or multiprocessing overweighs the speedups when the training context length is small. We leave efficient implementation of the parallel algorithm on a CUDA kernel for future work.

Conclusion We introduced adjoint sharding, a distributed and parallel computing algorithm, to facilitate training of LLMs on long contexts. Unlike the sequential backpropagation, the adjoint sharding computes gradients of each LLM layer against each token independently through vector-Jacobian product, allowing for parallel computation. To avoid the limitation of vjps increasing polynomially regarding context length, we propose truncated adjoint sharding to focus on important gradients. We analyzed the memory and FLOP cost of each computation block in adjoint sharding and proposed a method to accelerate it through parallel computing. Empirical results suggest orders of magnitude of memory reduction in training while maintaining the same training results as backpropagation.

References

- [1] Quentin Anthony, Yury Tokpanov, Paolo Glorioso, and Beren Millidge. Blackmamba: Mixture of experts for state-space models, 2024. URL <https://arxiv.org/abs/2402.01771>.
- [2] Randall Balestriero and Richard Baraniuk. Fast jacobian-vector product for deep networks, 2021. URL <https://arxiv.org/abs/2104.00219>.
- [3] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey, 2018. URL <https://arxiv.org/abs/1502.05767>.
- [4] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey, 2018. URL <https://arxiv.org/abs/1502.05767>.
- [5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory, 2024. URL <https://arxiv.org/abs/2405.04517>.
- [6] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [7] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, et al. Internlm2 technical report, 2024. URL <https://arxiv.org/abs/2403.17297>.
- [8] Yang Cao, Shengtai Li, and Linda Petzold. Adjoint sensitivity analysis for differential-algebraic equations: algorithms and software. *Journal of Computational and Applied Mathematics*, 149(1):171–191, 2002. ISSN 0377-0427. doi: [https://doi.org/10.1016/S0377-0427\(02\)00528-9](https://doi.org/10.1016/S0377-0427(02)00528-9). URL <https://www.sciencedirect.com/science/article/pii/S0377042702005289>. Scientific and Engineering Computations for the 21st Century - Methodologies and Applications Proceedings of the 15th Toyota Conference.
- [9] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations, 2019. URL <https://arxiv.org/abs/1806.07366>.
- [10] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation, 2023. URL <https://arxiv.org/abs/2306.15595>.
- [11] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models, 2024. URL <https://arxiv.org/abs/2309.12307>.
- [12] Saeed Damadi, Golnaz Moharrer, and Mostafa Cham. The backpropagation algorithm for a math student, 2023. URL <https://arxiv.org/abs/2301.09977>.
- [13] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [14] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- [15] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- [16] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [17] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024. URL <https://arxiv.org/abs/2402.19427>.
- [18] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. Longrope: Extending llm context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024.
- [19] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. Longrope: Extending llm context window beyond 2 million tokens, 2024. URL <https://arxiv.org/abs/2402.13753>.
- [20] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. Augmented neural odes, 2019. URL <https://arxiv.org/abs/1904.01681>.

- [21] Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models, 2023. URL <https://arxiv.org/abs/2212.14052>.
- [22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- [23] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- [24] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state-space layers, 2021. URL <https://arxiv.org/abs/2110.13985>.
- [25] Albert Gu, Isys Johnson, Aman Timalsina, Atri Rudra, and Christopher Ré. How to train your hippo: State space models with generalized orthogonal basis projections, 2022. URL <https://arxiv.org/abs/2206.12037>.
- [26] Meng-Hao Guo, Tian-Xing Xu, Jiang-Jiang Liu, Zheng-Ning Liu, Peng-Tao Jiang, Tai-Jiang Mu, Song-Hai Zhang, Ralph R Martin, Ming-Ming Cheng, and Shi-Min Hu. Attention mechanisms in computer vision: A survey. *Computational visual media*, 8(3):331–368, 2022.
- [27] Ankit Gupta, Harsh Mehta, and Jonathan Berant. Simplifying and understanding state space models with diagonal linear rnns, 2023. URL <https://arxiv.org/abs/2212.00768>.
- [28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- [29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [30] Julien Herrmann, Olivier Beaumont, Lionel Eyraud-Dubois, Julien Hermann, Alexis Joly, and Alena Shilova. Optimal checkpointing for heterogeneous chains: how to train deep neural networks with limited memory, 2019. URL <https://arxiv.org/abs/1911.13214>.
- [31] Herbert Jaeger. A tutorial on training recurrent neural networks , covering bppt , rtl , ekf and the ” echo state network ” approach - semantic scholar. In *National Research Center for Information Technology*, 2002, 2005. URL <https://api.semanticscholar.org/CorpusID:192593367>.
- [32] Steven Johnson. Adjoint methods and sensitivity analysis for recurrence, 01 2007.
- [33] Shiva Kaul. Linear dynamical systems as a core computational primitive. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 16808–16820. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/c3581d2150ff68f3b33b22634b8adaea-Paper.pdf.
- [34] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023.
- [35] Dacheng Li*, Rulin Shao*, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. How long can open-source llms truly promise on context length?, June 2023. URL <https://lmsys.org/blog/2023-06-29-longchat>.
- [36] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context llms struggle with long in-context learning, 2024. URL <https://arxiv.org/abs/2404.02060>.
- [37] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [38] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context, 2023. URL <https://arxiv.org/abs/2310.01889>.
- [39] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with blockwise ringattention, 2024. URL <https://arxiv.org/abs/2402.08268>.
- [40] Meta et al. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [41] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training, 2018. URL <https://arxiv.org/abs/1710.03740>.

- [42] Roman Novak, Jascha Sohl-Dickstein, and Samuel S. Schoenholz. Fast finite width neural tangent kernel, 2022. URL <https://arxiv.org/abs/2206.08720>.
- [43] NVIDIA. Matrix multiplication background user’s guide, 2024. URL <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [44] OpenAI et al. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- [45] Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences, 2023. URL <https://arxiv.org/abs/2303.06349>.
- [46] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks, 2013. URL <https://arxiv.org/abs/1211.5063>.
- [47] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [48] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.
- [49] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemysław Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanisław Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. Rwk: Reinventing rnns for the transformer era, 2023. URL <https://arxiv.org/abs/2305.13048>.
- [50] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models, 2023. URL <https://arxiv.org/abs/2309.00071>.
- [51] Maciej Pióro, Kamil Ciebia, Krystian Król, Jan Ludziejewski, Michał Krutul, Jakub Krajewski, Szymon Antoniuk, Piotr Miłoś, Marek Cygan, and Sebastian Jaszczur. Moe-mamba: Efficient selective state space models with mixture of experts, 2024. URL <https://arxiv.org/abs/2401.04081>.
- [52] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. URL <https://arxiv.org/abs/1910.02054>.
- [53] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. URL <https://arxiv.org/abs/1910.02054>.
- [54] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. Zero-offload: Democratizing billion-scale model training, 2021. URL <https://arxiv.org/abs/2101.06840>.
- [55] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL <https://arxiv.org/abs/2407.08608>.
- [56] Nimit S. Sohoni, Christopher R. Aberger, Megan Leszczynski, Jian Zhang, and Christopher Ré. Low-memory neural network training: A technical report, 2022. URL <https://arxiv.org/abs/1904.10631>.
- [57] Corentin Tallec and Yann Ollivier. Unbiasing truncated backpropagation through time, 2017. URL <https://arxiv.org/abs/1705.08209>.
- [58] Szymon Tworkowski, Konrad Staniszewski, Mikołaj Pacek, Yuhuai Wu, Henryk Michalewski, and Piotr Miłoś. Focused transformer: Contrastive training for context scaling, 2023. URL <https://arxiv.org/abs/2307.03170>.
- [59] Reddit users. Ntk-aware scaled rope, 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/141z7j5/ntkaware_scaled_rope_allows_llama_models_to_have/.
- [60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [61] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen, and Jan S. Rellermeyer. A survey on distributed machine learning. *ACM Computing Surveys*, 53(2):1–33, March 2020. ISSN 1557-7341. doi: 10.1145/3377454. URL <http://dx.doi.org/10.1145/3377454>.

- [62] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, Garvit Kulshreshtha, Vartika Singh, Jared Casper, Jan Kautz, Mohammad Shoeybi, and Bryan Catanzaro. An empirical study of mamba-based language models, 2024. URL <https://arxiv.org/abs/2406.07887>.
- [63] Shida Wang and Beichen Xue. State-space models with layer-wise nonlinearity are universal approximators with exponential decaying memory, 2023. URL <https://arxiv.org/abs/2309.13414>.
- [64] P.J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10): 1550–1560, 1990. doi: 10.1109/5.58337.
- [65] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2024. URL <https://arxiv.org/abs/2309.17453>.
- [66] Xingzi Xu, Ali Hasan, Khalil Elkhailil, Jie Ding, and Vahid Tarokh. Characteristic neural ordinary differential equations, 2022. URL <https://arxiv.org/abs/2111.13207>.
- [67] Jianwei Yang, Chunyuan Li, Pengchuan Zhang, Xiyang Dai, Bin Xiao, Lu Yuan, and Jianfeng Gao. Focal self-attention for local-global interactions in vision transformers, 2021. URL <https://arxiv.org/abs/2107.00641>.
- [68] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Soaring from 4k to 400k: Extending llm’s context with activation beacon, 2024. URL <https://arxiv.org/abs/2401.03462>.
- [69] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. Pytorch fsdp: Experiences on scaling fully sharded data parallel, 2023. URL <https://arxiv.org/abs/2304.11277>.

A Appendix

A.1 Proof for proposition 2

Proof 1 Define $\partial \tilde{\mathbf{y}} / \partial \mathbf{h}^t = \tilde{\mathbf{y}}_{\mathbf{h}^t}^t$, $\partial \tilde{\mathbf{h}}^t / \partial \mathbf{h}^{t-1} = \tilde{\mathbf{h}}_{\mathbf{h}^{t-1}}^t$, and $\partial \tilde{\mathbf{y}} / \partial \boldsymbol{\theta} = \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t$, $\partial \tilde{\mathbf{h}}^t / \partial \boldsymbol{\theta} = \tilde{\mathbf{h}}_{\boldsymbol{\theta}}^t$, by plugging in the expression for $\tilde{\mathbf{y}}^t$ from subsection 3.2, proposition 1 states that

$$\frac{d\tilde{\mathbf{y}}^t}{d\boldsymbol{\theta}} = \tilde{\mathbf{y}}_{\mathbf{h}^t}^t \left[\left(\prod_{i=1}^{t-1} \mathbf{h}_{\mathbf{h}^{t-i}}^{t-i+1} \right) \mathbf{h}_{\boldsymbol{\theta}}^1 + \left(\prod_{i=1}^{t-2} \mathbf{h}_{\mathbf{h}^{t-i}}^{t-i+1} \right) \mathbf{h}_{\boldsymbol{\theta}}^2 + \cdots + \mathbf{h}_{\mathbf{h}^{t-1}}^t \mathbf{h}_{\boldsymbol{\theta}}^{t-1} + \mathbf{h}_{\boldsymbol{\theta}}^t \right] + \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t.$$

In the context of SSM, we have:

$$\mathbf{h}^t = \mathbf{A}^t \mathbf{h}^{t-1} + \mathbf{B}^t \hat{\mathbf{x}}^t, \mathbf{h}_{\mathbf{h}^{t-1}}^t = \mathbf{A}^t, \mathbf{h}_{\boldsymbol{\theta}}^t = \mathbf{A}_{\boldsymbol{\theta}}^t \mathbf{h}^{t-1} + \mathbf{B}_{\boldsymbol{\theta}}^t \hat{\mathbf{x}}^t, \tilde{\mathbf{y}}^t = \mathbf{C}^t \mathbf{h}^t, \tilde{\mathbf{y}}_{\mathbf{h}^t}^t = \mathbf{C}^t, \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t = \mathbf{C}_{\boldsymbol{\theta}}^t \mathbf{h}^t. \quad (8)$$

Plugging in these relations, we get:

$$\frac{d\tilde{\mathbf{y}}^t}{d\boldsymbol{\theta}} = \mathbf{C}^t \left[\left(\prod_{i=1}^{t-1} \mathbf{A}^{t+1-i} \right) \mathbf{h}_{\boldsymbol{\theta}}^1 + \left(\prod_{i=1}^{t-2} \mathbf{A}^{t+1-i} \right) \mathbf{h}_{\boldsymbol{\theta}}^2 + \cdots + \left(\prod_{i=1}^2 \mathbf{A}^{t+1-i} \right) \mathbf{h}_{\boldsymbol{\theta}}^{t-2} + \mathbf{A}^t \mathbf{h}_{\boldsymbol{\theta}}^{t-1} + \mathbf{h}_{\boldsymbol{\theta}}^t \right] + \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t. \quad (9)$$

Define the adjoint state $\boldsymbol{\lambda}^{t,\tau} = \mathbf{C}^t (\prod_{i=1}^{t-\tau} \mathbf{A}^{t+1-i})$, we have

$$\frac{d\tilde{\mathbf{y}}^t}{d\boldsymbol{\theta}} = \boldsymbol{\lambda}^{t,1} \mathbf{h}_{\boldsymbol{\theta}}^1 + \boldsymbol{\lambda}^{t,2} \mathbf{h}_{\boldsymbol{\theta}}^2 + \cdots + \boldsymbol{\lambda}^{t,t-1} \mathbf{h}_{\boldsymbol{\theta}}^{t-1} + \boldsymbol{\lambda}^{t,t} \mathbf{h}_{\boldsymbol{\theta}}^t + \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t$$

Therefore, we have

$$\begin{aligned} \frac{d\mathbf{l}^t}{d\boldsymbol{\theta}} &= \frac{d\mathbf{l}^t}{d\mathbf{y}^t} \frac{d(\mathbf{y}^t + \hat{\mathbf{x}}^t)}{d\boldsymbol{\theta}} \\ &= \frac{d\mathbf{l}^t}{d\mathbf{y}^t} \frac{d\tilde{\mathbf{y}}^t}{d\boldsymbol{\theta}} \\ &= \frac{d\mathbf{l}^t}{d\mathbf{y}^t} [\boldsymbol{\lambda}^{t,1} \mathbf{h}_{\boldsymbol{\theta}}^1 + \boldsymbol{\lambda}^{t,2} \mathbf{h}_{\boldsymbol{\theta}}^2 + \cdots + \boldsymbol{\lambda}^{t,t-1} \mathbf{h}_{\boldsymbol{\theta}}^{t-1} + \boldsymbol{\lambda}^{t,t} \mathbf{h}_{\boldsymbol{\theta}}^t + \tilde{\mathbf{y}}_{\boldsymbol{\theta}}^t] \end{aligned}$$

Plug in everything, we have

$$\begin{aligned}
\frac{dl^t}{d\theta} &= \frac{dl^t}{dy^t} [\lambda^{t,1}(\mathbf{A}_\theta^1 \mathbf{h}^0 + \mathbf{B}_\theta^1 \hat{\mathbf{x}}^1) + \lambda^{t,2}(\mathbf{A}_\theta^2 \mathbf{h}^1 + \mathbf{B}_\theta^2 \hat{\mathbf{x}}^2) + \dots + \lambda^{t,t}(\mathbf{A}_\theta^t \mathbf{h}^{t-1} + \mathbf{B}_\theta^t \hat{\mathbf{x}}^t) + \mathbf{C}_\theta^t \mathbf{h}^t] \\
&= \left[\sum_{i=1}^t \frac{dl^t}{dy^t} \lambda^{t,i} (\mathbf{A}_\theta^i \mathbf{h}^{i-1} + \mathbf{B}_\theta^i \hat{\mathbf{x}}^i) \right] + \frac{dl^t}{dy^t} \mathbf{C}_\theta^t \mathbf{h}^t \\
&= \left[\sum_{i=1}^t \text{vjp}_{\mathcal{A}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1} \right) + \text{vjp}_{\mathcal{B}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i \right) \right] + \text{vjp}_{\mathcal{C}^t} \left(\frac{dl^t}{dy^t} \otimes \mathbf{h}^t \right)
\end{aligned}$$

where we define $\text{vjp}_{NN^i}(v) = v \cdot NN_\theta(\text{Input}^i)$, with θ being NN 's parameters and i being the index of Input. Now, as $\text{vjp}_{\mathcal{A}^i}(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1})$, $\text{vjp}_{\mathcal{B}^i}(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i)$, and $\text{vjp}_{\mathcal{C}^t}(\frac{dl^t}{dy^t} \otimes \mathbf{h}^t)$ are separate, we have

$$\frac{dl^t}{d\theta} = \left[\sum_{i=1}^t \text{vjp}_{\mathcal{A}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1} \right) \right] \oplus \left[\sum_{i=1}^t \text{vjp}_{\mathcal{B}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i \right) \right] \oplus \text{vjp}_{\mathcal{C}^t} \left(\frac{dl^t}{dy^t} \otimes \mathbf{h}^t \right), \quad (10)$$

where $\oplus$ is vector concatenation.

A.2 Proof for proposition 3

Proof 2 First, using the structure of ResNet, we have

$$\begin{aligned}
\frac{dL}{d\theta} &= \sum_{t=1}^T \frac{dl^t}{dy_K^t} \frac{dy_K^t}{d\theta} \\
&= \sum_{t=1}^T \frac{dl^t}{dy_K^t} \frac{d(\mathbf{y}_0^t + \sum_{k=1}^K \tilde{\mathbf{y}}_k^t)}{d\theta} \\
&= \sum_{t=1}^T \frac{dl^t}{dy_K^t} \sum_{k=1}^K \frac{d\tilde{\mathbf{y}}_k^t}{d\theta} \\
&= \sum_{t=1}^T \sum_{k=1}^K \frac{dl^t}{dy_K^t} \frac{d\tilde{\mathbf{y}}_k^t}{d\theta}
\end{aligned}$$

from proposition 2, we have proven that for a single SSM model, we have

$$\frac{dl^t}{d\theta} = \left[\sum_{i=1}^t \text{vjp}_{\mathcal{A}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \mathbf{h}^{i-1} \right) \right] \oplus \left[\sum_{i=1}^t \text{vjp}_{\mathcal{B}^i} \left(\frac{dl^t}{dy^t} \lambda^{t,i} \otimes \hat{\mathbf{x}}^i \right) \right] \oplus \text{vjp}_{\mathcal{C}^t} \left(\frac{dl^t}{dy^t} \otimes \mathbf{h}^t \right),$$

so for the ResNet model, we have

$$\begin{aligned}
\frac{dL}{d\theta} &= \sum_{t=1}^T \sum_{k=1}^K \frac{dl^t}{dy_K^t} \frac{d\tilde{y}_k^t}{d\theta} \\
&= \sum_{t=1}^T \sum_{k=1}^K \left\{ \left[\sum_{i=1}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) \right] \oplus \left[\sum_{i=1}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{x}}_k^i \right) \right] \oplus \text{vjp}_{\mathcal{C}_k^t} \left(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t \right) \right\} \\
&= \left(\sum_{t=1}^T \sum_{k=1}^K \text{vjp}_{\mathcal{C}_k^t} \left(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t \right) \right) \\
&\oplus \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) \right) \\
&\oplus \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{x}}_k^i \right) \right) \\
&= \left(\sum_{t=1}^T \sum_{k=1}^K \text{vjp}_{\mathcal{C}_k^t} \left(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t \right) \right) \\
&\oplus \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{A}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1} \right) \right) \\
&\oplus \left(\sum_{t=1}^T \sum_{k=1}^K \sum_{i=1}^t \text{vjp}_{\mathcal{B}_k^i} \left(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i \right) \right)
\end{aligned} \tag{11}$$

where the input to $\text{vjp}_{\mathcal{C}_k^t}(\frac{dl^t}{dy_K^t} \otimes \mathbf{h}_k^t)$, $\text{vjp}_{\mathcal{A}_k^i}(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \mathbf{h}_k^{i-1})$, and $\text{vjp}_{\mathcal{B}_k^i}(\frac{dl^t}{dy_K^t} \lambda_k^{t,i} \otimes \hat{\mathbf{y}}_{k-1}^i)$ are computed with the k -th SSM and the $\hat{\mathbf{x}}_k^i = \hat{\mathbf{y}}_{k-1}^i = \text{RMSNorm}(\mathbf{y}_{k-2}^i + \text{SSM}_{k-1}(\hat{\mathbf{Y}}_{k-2})^i)$ (the normalized output sequence of the $(k-1)$ -th SSM), and the adjoint state $\lambda_k^{t,\tau} = \mathbf{C}_k^t (\prod_{i=1}^{t-\tau} \mathbf{A}_k^{t+1-i})$.

A.3 Proof of concept for VJP

As a proof of concept of why $(dl^t/dy^t)\mathbf{C}_\theta^t \mathbf{h}^t$ can be computed with vjp, we present an explicit and simple example. We have $\mathbf{y} = [y_1, y_2]$, $\mathbf{h} = [h_1, h_2, h_3]$, $\theta = \bar{\theta}$. We then have

$$\begin{aligned}
\frac{dl}{dy} &= [l_{y_1} \quad l_{y_2}] \in \mathbb{R}^{1 \times P} \\
\mathbf{C}_\theta &= \begin{bmatrix} C_{11}^{\bar{\theta}} & C_{12}^{\bar{\theta}} & C_{13}^{\bar{\theta}} \\ C_{21}^{\bar{\theta}} & C_{22}^{\bar{\theta}} & C_{23}^{\bar{\theta}} \end{bmatrix} \in \mathbb{R}^{P \times N \times |\theta|} \\
\mathbf{h} &= \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \in \mathbb{R}^{N \times 1}
\end{aligned}$$

With each $C_{ij}^{\bar{\theta}} = [\partial C_{ij} / \partial \theta_1, \dots, \partial C_{ij} / \partial \theta_{|\theta|}] \in \mathbb{R}^{|\theta|}$. We have

$$\begin{aligned}
\frac{dl}{dy} \mathbf{C}_\theta \mathbf{h} &= C_{11}^{\bar{\theta}} l_{y_1} h_1 + C_{21}^{\bar{\theta}} l_{y_2} h_1 + C_{12}^{\bar{\theta}} l_{y_1} h_2 + C_{22}^{\bar{\theta}} l_{y_2} h_2 + C_{13}^{\bar{\theta}} l_{y_1} h_3 + C_{23}^{\bar{\theta}} l_{y_2} h_3 \\
&= [l_{y_1} h_1 \quad l_{y_1} h_2 \quad l_{y_1} h_3 \quad l_{y_2} h_1 \quad l_{y_2} h_2 \quad l_{y_2} h_3] \cdot [C_{11}^{\bar{\theta}} \quad C_{12}^{\bar{\theta}} \quad C_{13}^{\bar{\theta}} \quad C_{21}^{\bar{\theta}} \quad C_{22}^{\bar{\theta}} \quad C_{23}^{\bar{\theta}}] \\
&= \text{sum} \left(\left(\begin{bmatrix} l_{y_1} \\ l_{y_2} \end{bmatrix} \otimes [h_1 \quad h_2 \quad h_3] \right) \circ \begin{bmatrix} C_{11}^{\bar{\theta}} & C_{12}^{\bar{\theta}} & C_{13}^{\bar{\theta}} \\ C_{21}^{\bar{\theta}} & C_{22}^{\bar{\theta}} & C_{23}^{\bar{\theta}} \end{bmatrix} \right)
\end{aligned}$$

where $\cdot$ is vector dot product, $\otimes$ is vector outer product, $\circ$ is element-wise product, and sum means summing all elements in a matrix.

A.4 Distributed tensors' locations

We provide the specific location for each tensors in distributed training:

Table 2: Tensors stored on each GPU, part 1.

GPU index	$dl(\mathbf{o}^t)/dy_K^t$	h_k^t
$v = 1$	$t = 1, \dots, T$	$t = 1, \dots, T; k = 1, \dots, K//\Upsilon$
$v = 2$	$t = 1, \dots, T$	$t = 1, \dots, T; k = K//\Upsilon + 1, \dots, 2(K//\Upsilon)$
$\dots$	$\dots$	$\dots$
$v = \Upsilon - 1$	$t = 1, \dots, T$	$t = 1, \dots, T; k = (\Upsilon - 2)(K//\Upsilon) + 1, \dots, (\Upsilon - 1)(K//\Upsilon)$
$v = \Upsilon$	$t = 1, \dots, T$	$t = 1, \dots, T; k = (\Upsilon - 1)(K//\Upsilon) + 1, \dots, K$

Table 3: Tensors stored on each GPU, part 2.

GPU index	C_k^t
$v = 1$	$t = 1, \dots, T; k = 1, \dots, K//\Upsilon$
$v = 2$	$t = 1, \dots, T; k = K//\Upsilon + 1, \dots, 2(K//\Upsilon)$
$\dots$	$\dots$
$v = \Upsilon - 1$	$t = 1, \dots, T$
$v = \Upsilon$	$t = 1, \dots, T; k = (\Upsilon - 1)(K//\Upsilon) + 1, \dots, K$

Table 4: Tensors stored on each GPU, part 3.

GPU index	$\hat{g}_k^t$
$v = 1$	$t = 1, \dots, T; k = 0, \dots, K//\Upsilon - 1$
$v = 2$	$t = 1, \dots, T; k = K//\Upsilon, \dots, 2(K//\Upsilon) - 1$
$\dots$	$\dots$
$v = \Upsilon - 1$	$t = 1, \dots, T; k = (\Upsilon - 2)(K//\Upsilon), \dots, (\Upsilon - 1)(K//\Upsilon) - 1$
$v = \Upsilon$	$t = 1, \dots, T; k = (\Upsilon - 1)(K//\Upsilon), \dots, K - 1$

Table 5: Tensors stored on each GPU, part 4.

GPU index	A_k^t
$v = 1$	$t = 2, \dots, T; k = 1, \dots, K//\Upsilon$
$v = 2$	$t = 2, \dots, T; k = K//\Upsilon + 1, \dots, 2(K//\Upsilon)$
$\dots$	$\dots$
$v = \Upsilon - 1$	$t = 2, \dots, T; k = (\Upsilon - 2)(K//\Upsilon) + 1, \dots, (\Upsilon - 1)(K//\Upsilon)$
$v = \Upsilon$	$t = 2, \dots, T; k = (\Upsilon - 1)(K//\Upsilon) + 1, \dots, K$

Table 6: Tensors stored on each GPU, part 5.

GPU index	θ_k	Gradient $_k$
$v = 1$	$k = 1, \dots, K//\Upsilon$	$k = 1, \dots, K//\Upsilon$
$v = 2$	$k = K//\Upsilon + 1, \dots, 2(K//\Upsilon)$	$k = K//\Upsilon + 1, \dots, 2(K//\Upsilon)$
$\dots$	$\dots$	$\dots$
$v = \Upsilon - 1$	$k = (\Upsilon - 2)(K//\Upsilon) + 1, \dots, (\Upsilon - 1)(K//\Upsilon)$	$k = (\Upsilon - 2)(K//\Upsilon) + 1, \dots, (\Upsilon - 1)(K//\Upsilon)$
$v = \Upsilon$	$k = (\Upsilon - 1)(K//\Upsilon) + 1, \dots, K$	$k = (\Upsilon - 1)(K//\Upsilon) + 1, \dots, K$